THE UNIVERSITY OF WESTERN AUSTRALIA

Client-Side Rendering

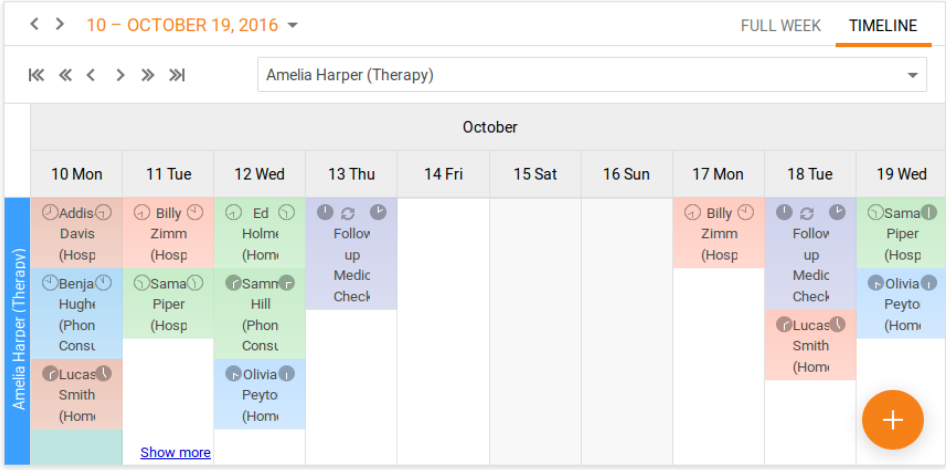
CITS3403 and CITS5505 - Agile Web Development

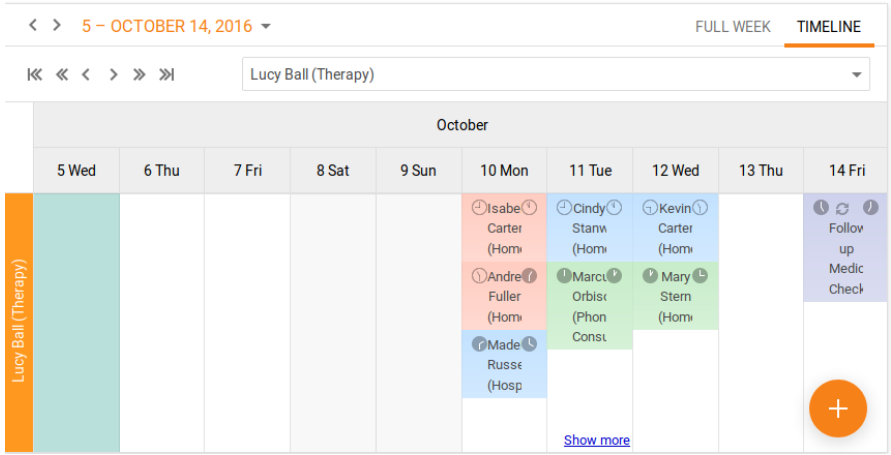
Lecturer: Matthew Daggitt

Semester 1, 2026

Why use client-side rendering?

- So far, the JavaScript we have seen responds to local events in the browser, such as users clicking buttons, pages loading, and mouse movements.
- However, we often want to respond to remote events, such as someone sending you a message, liking a post etc.
- We also may want to dynamically respond to a local event using information on the server: if a user enters the 1st of April as a preferred appointment date, then we would like to immediately show them the available appointments.
- We could send the date to the server, have the server rebuild the page and send the entire page back, but we only require a few bytes of data!

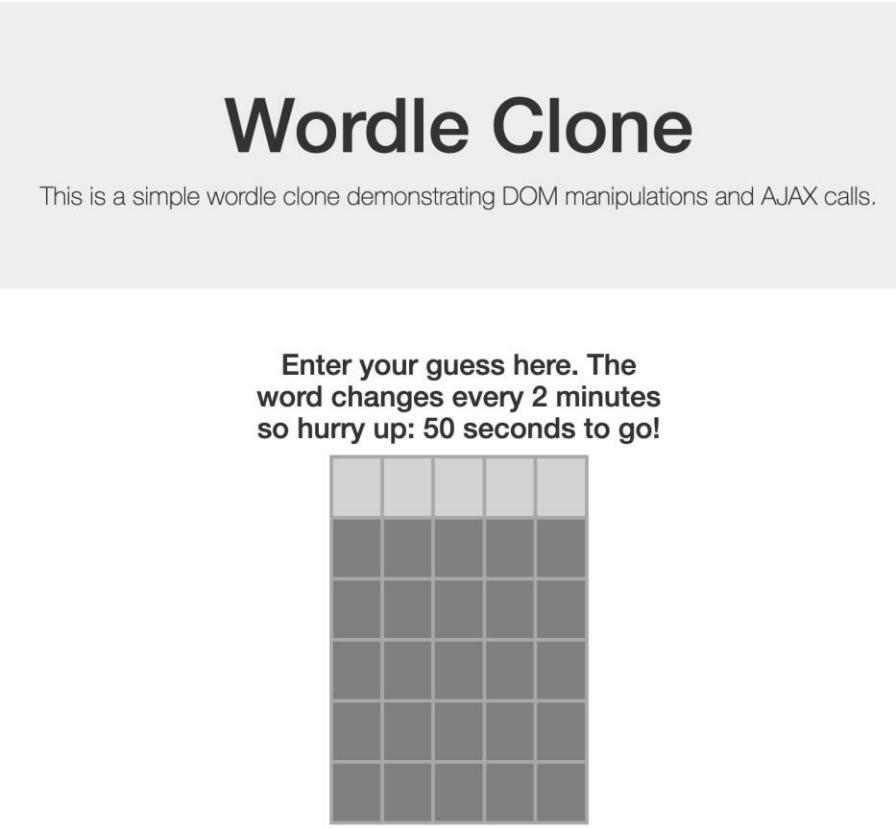




Client-side rendering with AJAX

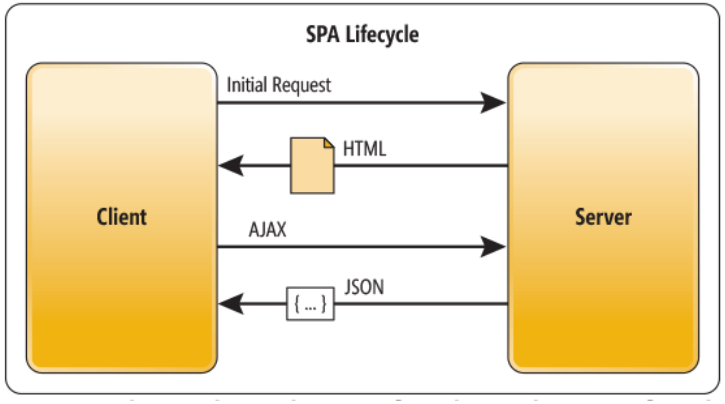
Wordle example

- As a simple example of client-side rendering we will look at a simple clone of the popular word game Wordle.
- It will use AJAX to send and receive requests from the server.
- It will use JavaScript and the DOM to update the web page.
- Available at: https://drtnf.net/static/wordle.html



Initial static files

- When using client-side rendering we still need to send initial HTML/CSS/JS code.



- Unlike with server-side rendering, the same initial template is sent to every client. Therefore, we can use static files.

Serving static files in Flask

- Flask projects have a directory called "static" to serve static files, including HTML, CSS, JS and images.

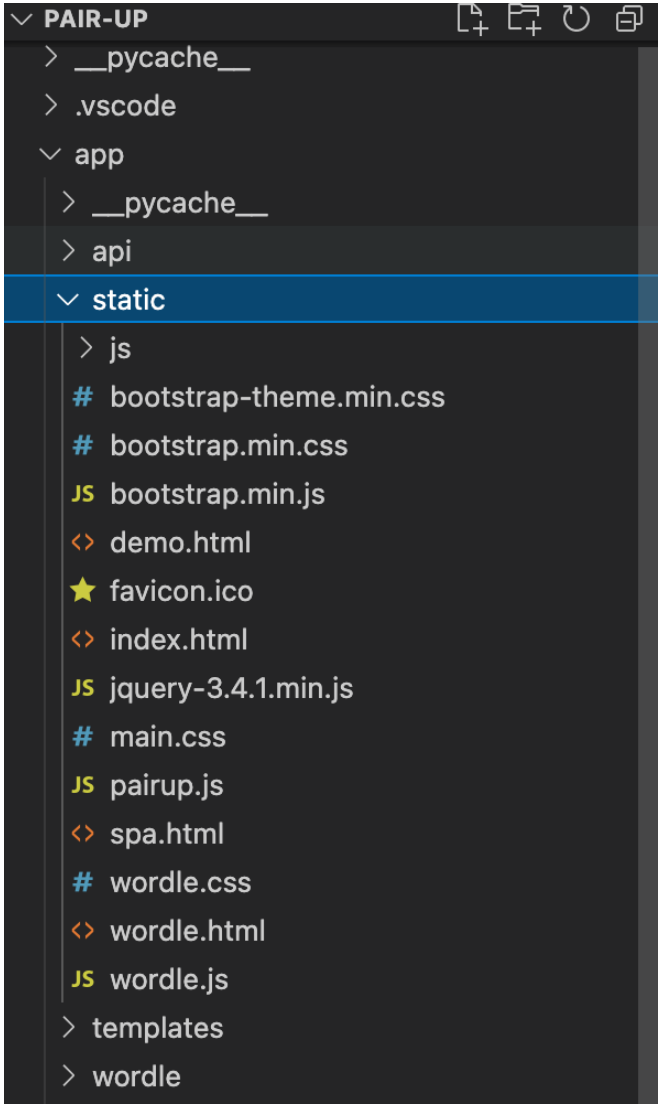
- Flask automatically creates an endpoint called `static` with the following route:

  `/static/<path:filename>`

  which loads resources from this folder.

- We can then have Flask redirect requests to a given route, to the static files we want to serve using `url_for`.

```
@app.route('/speed_wordle')
def speed_wordle():
  return redirect(url_for('static', filename='wordle.html'))
```



Frontend: initial static HTML file

- The initial HTML skeleton is very minimal.

```html
<html>
  <head>
    <meta charset="utf8">
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <!-- Missing imports for boostrap v3.4.1 and JQuery v3.6.0 -->

    <script src="./wordle.js"></script>
    <link rel="stylesheet" href="./wordle.css">
  </head>

  <body>
    <div class="container">
      <div class="jumbotron">
        <h1>Wordle Clone</h1>
        <p>Wordle clone demonstrating DOM manipulations and AJAX calls.</p>
      </div>
      <div class="container col-sm-4"></div>
      <div class="container col-sm-4">
        <h3> Enter your guess here. The word changes every 2 minutes so hurry up:
          <span id="time_left"></span> seconds to go!
        </h3>
        <table class='wordle' id="guesses"></table>
      </div>
      <div class="container col-sm-4"></div>
    </div>
```

Frontend: static JavaScript files

- We can initialise HTML representation of the table of guesses as follows.
- We dynamically create the rows and column cells and store references to the elements in the `table_data` variable so we can update them later.

```javascript
let table_data = [];
let current_guess = 0;
let current_cell = 0;


function init(){
  let table = document.getElementById("guesses");
  table.innerHTML="";
  let tbody = document.createElement("tbody");
  for(let i = 0; i<6; i++){
    let row_data = [];
    let row = document.createElement("TR");
    for(let j = 0; j<5; j++){
      let cell = document.createElement("TD");
      cell.innerHTML="   ";
      row.appendChild(cell);
      row_data[j] = cell;
    }
    table_data[i] = row_data;
    tbody.appendChild(row);
  }
  tbody.children[0].classList.add("active");
  table.appendChild(tbody);
  current_guess = 0;
  current_cell = 0;
  getTimeLeft();
  document.getElementById("close").addEventListener("click", function(){
    document.getElementById("end_game").style.display = 'none';
  });
}
```

Frontend: Making requests for time left

- The first example of client-side rendering is a simple request to get the time left for the current puzzle. This is a one-off request when the page is loaded.

```javascript
function getTimeLeft(){
    const xhttp = new XMLHttpRequest();
    xhttp.open("GET", "https://drtnf.net/wordle_time_left", true);
    xhttp.onload = function(e) {
      time_left = JSON.parse(xhttp.responseText).time_left;
      let x = setInterval(function() {
        document.getElementById("time_left").innerHTML = time_left--;
        if(time_left<0){
          clearInterval(x);
          init();
        }
      }, 1000);

    };
    xhttp.send();
}

```

Backend: Requests for time left

- When we receive a time left request, we respond to the request with a JSON object with a single field '`time_left`'.

```
'''
Gives time remaining for the current puzzle
'''
@app.route('/wordle_time_left', methods=['GET'])
def wordle_time_left():
    response = jsonify({'time_left':check_time()})
    response.status_code = 201
    return response

```

- We must now implement the logic for checking the time left.

Backend: Requests for time left

```python
update_delta = 120
words = []
with open('./app/wordle/fives.txt','r') as word_file:
    for word in word_file:
        words.append(word[:-1])


'''
Renews the selected word every update delta seconds
'''
def check_time():
  f = open('./app/wordle/last_update.txt','r')
  last_update = int(f.read())
  f.close()
  now = int(time.time())
  if (now-last_update) > update_delta:
      last_update = now
      secret = words[random.randrange(len(words))]
      f = open('./app/wordle/answer.txt','w')
      f.write(secret)
      f.close
      f = open('./app/wordle/last_update.txt','w')
      f.write(str(last_update));
      f.close
  return update_delta-(now-last_update)
```

- To respond consistently we need to persist the state, i.e. the current word and when it was created.
- As we do not yet have a database, we can store the time since the word being guessed was updated in a text file `last_update.txt`.
- If it is time for a new word, we write the new word into another text file 'answer.txt`.

Frontend: Entering guesses

- In the JavaScript file we can listen and respond to key presses as follows:

```javascript
function isAlpha(c){
  return /^[A-Z]$/i.test(c);
}

document.addEventListener("keydown", evt =>{
  let key = evt.key;
  if(key.length==1 && isAlpha(key) && current_cell<5 && current_guess<6){
    table_data[current_guess][current_cell].innerHTML=key.toUpperCase();
    current_cell++;
  }
  else if((key=="Delete" || key == "Backspace") && current_cell>0 && current_guess<6){
    current_cell--;
    table_data[current_guess][current_cell].innerHTML="   ";
  }
  else if(key == "Enter" && current_cell == 5 && current_guess<6){
    let guess="";
    for(let i = 0; i<5; i++){
      guess = guess + table_data[current_guess][i].innerHTML;
    }
    const xhttp = new XMLHttpRequest();

    xhttp.open("GET", "https://drtnf.net/wordle_guess?guess="+guess, true);
```

- When the user presses "enter" we extract the current guess from the page and then we submit it to the server, passing it as a parameter in the URL.

Backend: Responding to guess requests

- In the Flask backend when we receive a guess, we first check that the guess is valid and throw an appropriate error if not.

```
'''
route for handling wordle guesses
'''
@app.route('/wordle_guess',methods=['POST','GET'])
def wordle_guess():
    check_time()
    data = request.args or {}
    if 'guess' not in data or not data['guess'].isalpha() or len(data['guess']) != 5:
        return bad_request('Guess must be a five letter word')
    f = open('./app/wordle/answer.txt','r')
    secret = f.read()
    f.close()
    response = jsonify({'outcome':wordle(data['guess'].upper(), secret.upper())})
    response.status_code = 201
    return response
```

- We then consult the secret answer stored in the text file and compute the array of answers using the "wordle" function.

- Finally, we encode the answer array in a JSON object and send it as part of the response.

Backend: Guess logic

- What is missing is to implement the actual guess logic.

```
def wordle(guess, target):
    answer = [0]*5 #to return to user
    target_free = [True]*5  #for handling multiple letters
    for i in range(5):
        if guess[i]==target[i]:
            answer[i] = 2
            target_free[i] = False
    for i, c in enumerate(guess):
        for j, d in enumerate(target):
            if c==d and target_free[j] and answer[i]==0:
                answer[i] = 1
                target_free[j] = False
    return answer
```

- The function computes what information we should send back to the player about their guess.

- We use `2` to represent the right letter, right place and `1` to represent right letter, wrong place and `0` to represent wrong letter.

Frontend: Updating the UI after a guess

```
72      xhttp.onload = function(e) {
73        let result = JSON.parse(xhttp.responseText).outcome;
74        let sum = 0
75        for(let i = 0; i<5; i++){
76          if(result[i]==2){
77            sum+=result[i];
78            table_data[current_guess][i].classList.add('correct');
79          }
80          if(result[i]==1){
81            table_data[current_guess][i].classList.add('misplaced');
82          }
83        }
84        let tbody = document.getElementById("guesses").firstChild;
85        tbody.children[current_guess++].classList.remove('active');
86        if(sum==10){
87          document.getElementById('end_game').style.display="block";
88          document.getElementById('congrats').innerHTML="Congratulations!";
89        }
90        else{
91          current_cell = 0;
92          if(current_guess>5){
93            document.getElementById('end_game').style.display="block";
94            document.getElementById('congrats').innerHTML="Out of guesses!";
95          }
96          else tbody.children[current_guess].classList.add('active');
97        }
98      }
99      xhttp.send();
100   }
101 });
```

- Back in the JavaScript file we can then get the data by parsing the response text as JSON.
- We then update the page, adding CSS styling dynamically via the `classList` property to indicate the status of each letter of the guess.
- Finally, we check if the game is ended or not and update the page accordingly.
- We now have everything to make a fully functioning Wordle game!

Client-side vs server-side rendering

- What are the pros and cons of client-side vs server-side rendering?

1. Client-side rendering requires custom protocols – in our Wordle example, the server and the client had to use their own protocol to communicate the guess result (`2` = right letter, wrong place and `1` = right letter, wrong place, '0' = wrong).

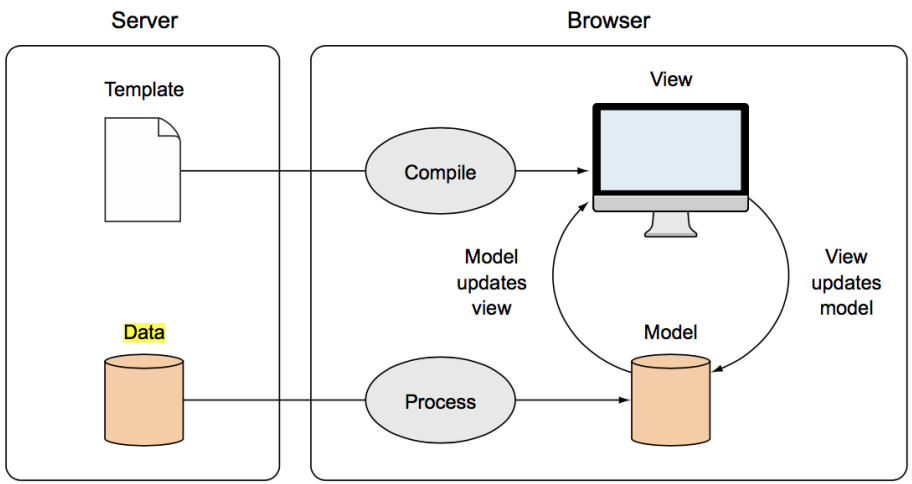
2. Client-side rendering is more efficient - we only had to send a single string to the server and receiving an array of 5 numbers back. In contrast server-side rendering would have had to send a whole page each time.

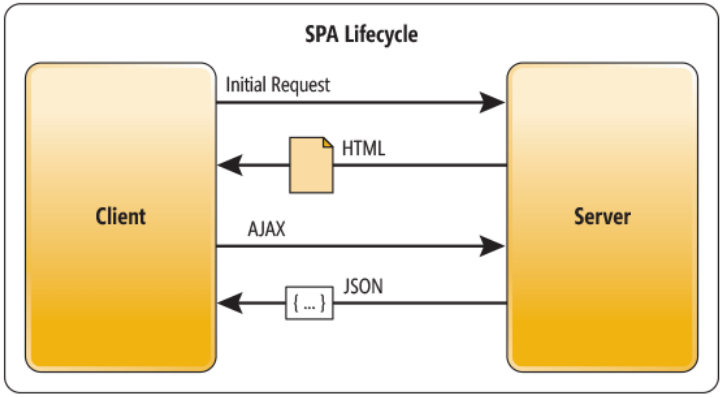
3. Client-side rendering requires far more complex logic on the client-side – had to store data and write several lengthy JavaScript functions to edit the page, without the support of the templating libraries we can use for server-side rendering.

Single page applications

Single-page applications (SPA)

- Single Page Applications are services where the entire website is provided via client-side rendering:
  - The browser/client do the heavy lifting i.e. logic and rendering.
  - The server just provides the data.
  - The user never navigates to a new URL, even when they move to what looks like a new page.
  - The LMS is an example of this.





Advantages and disadvantages

Pros of SPA

- Less load on the server, able to respond to more clients.
- A more responsive client. No need to wait for server responses.
- Genuine separation between content and presentation.

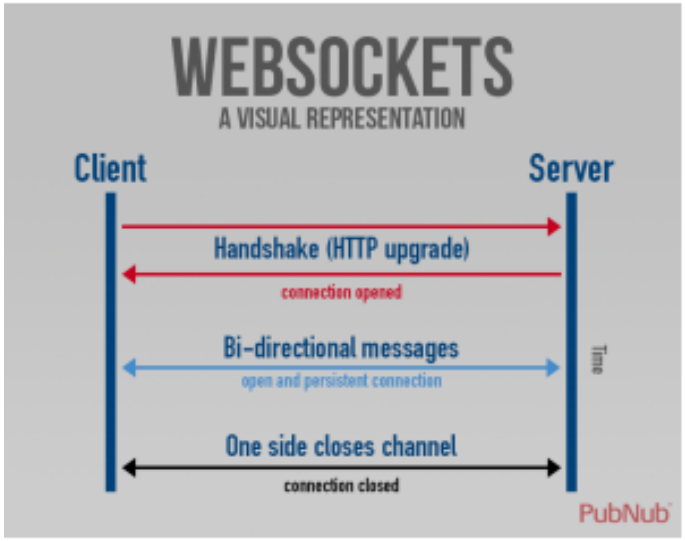
Cons of SPA

- Longer load time. A lot of JS must be transferred.
- Search engine optimisation (SEO) can be a problem. Robots won’t crawl JavaScript.
- Navigation (e.g. forward and back buttons) can be an issue.

Client-side rendering with web-sockets

Web-socket basics

- HTTP requests are useful for providing dynamic content but are heavy weight and expensive to setup.
- Many web applications depend on real time interaction.
- Web-sockets were standardised in 2011 to provide full duplex communication.
- Web-sockets allow your client-side JavaScript to open a persistent connection (stream) to the server.
- This allows real time communication in the application without having to send HTTP requests.



Web-sockets in Flask

- Web-sockets are supported in Flask via the `flask-socketIO` package

  https://flask-socketio.readthedocs.io/en/latest/

- SocketIO is good for message passing chat or distributed games.

- For direct video and audio, WebRTC can be used (peer-to-peer).

- Clients can connect to a socket on a server, and then the server can push messages to clients.

- The client has a *listener* architecture so it will respond to the push immediately.

Structuring a socket-based application

- Sockets mirror the routes architecture of a Flask project, but instead of listening for requests, they listen for messages and actions, and broadcast to all listening clients.

- The server works as a common blackboard for the session (or room) and the clients implement a listening architecture via jQuery.

- The socketIO architecture maintains rooms that users/processes can subscribe to.

- Clients and server interact by emitting events including *join, status, message,* and *leave.* You can also create customised events for clients to create and receive.

- We will follow a simple demonstration from Miguel Grinberg taken from:

https://github.com/miguelgrinberg/Flask-SocketIO-Chat

Web-sockets on the client-side

- We can use jQuery to send events to the server, listen for events coming from the server, and update the DOM accordingly.

```
40      <body>
41          <h1>Flask-SocketIO-Chat: {{ room }}</h1>
42          <textarea id="chat" cols="80" rows="20"></textarea><br><br>
43          <input id="text" size="80" placeholder="Enter your message here"><br><br>
44          <a href="#" onclick="leave_room();">Leave this room</a>
45      </body>
```

Web-sockets on the client-side

- The code below creates a socket and then listens for incoming messages.
- When can then listen for various events such as `connect`, `status` and `message`

```
2       <head>
3           <title>Flask-SocketIO-Chat: {{ room }}</title>
4           <script type="text/javascript" src="//code.jquery.com/jquery-1.4.2.min.js"></script>
5           <script type="text/javascript" src="//cdnjs.cloudflare.com/ajax/libs/socket.io/1.3.6/socket.io.min.js"></script>
6           <script type="text/javascript" charset="utf-8">
7               var socket;
8               $(document).ready(function(){
9                   socket = io.connect('http://' + document.domain + ':' + location.port + '/chat');
10                  socket.on('connect', function() {
11                      socket.emit('joined', {});
12                  });
13                  socket.on('status', function(data) {
14                      $('#chat').val($('#chat').val() + '<' + data.msg + '>\n');
15                      $('#chat').scrollTop($('#chat')[0].scrollHeight);
16                  });
17                  socket.on('message', function(data) {
18                      $('#chat').val($('#chat').val() + data.msg + '\n');
19                      $('#chat').scrollTop($('#chat')[0].scrollHeight);
20                  });
```

Web-sockets on the client-side

- We can send messages using the `socket.emit` method.
- We can disconnect using `socket.disconnect`

```
21                  $('#text').keypress(function(e) {
22                      var code = e.keyCode || e.which;
23                      if (code == 13) {
24                          text = $('#text').val();
25                          $('#text').val('');
26                          socket.emit('text', {msg: text});
27                      }
28                  });
29              });
30              function leave_room() {
31                  socket.emit('left', {}, function() {
32                      socket.disconnect();
33
34                      // go back to the login page
35                      window.location.href = "{{ url_for('main.index') }}";
36                  });
37              }
38          </script>
39      </head>
```

Web-sockets on the server-side

- We use a similar architecture to routes: A main folder called *main*, containing:
  - *forms.py* for registration,
  - *routes.py* for handling login
  - *events.py* for handling the socket events.

- The SocketIO package includes a decorator to match incoming messages with python methods.

```python
from flask import session
from flask_socketio import emit, join_room, leave_room
from .. import socketio


@socketio.on('joined', namespace='/chat')
def joined(message):
    """Sent by clients when they enter a room.
    A status message is broadcast to all people in the room."""
    room = session.get('room')
    join_room(room)
    emit('status', {'msg': session.get('name') + ' has entered the room.'}, room=room)


@socketio.on('text', namespace='/chat')
def text(message):
    """Sent by a client when the user entered a new message.
    The message is sent to all people in the room."""
    room = session.get('room')
    emit('message', {'msg': session.get('name') + ':' + message['msg']}, room=room)


@socketio.on('left', namespace='/chat')
def left(message):
    """Sent by clients when they leave a room.
    A status message is broadcast to all people in the room."""
    room = session.get('room')
    leave_room(room)
    emit('status', {'msg': session.get('name') + ' has left the room.'}, room=room)

```

Other applications of web-sockets

- Sockets can be used for distributing real time events such as real-time scoreboards or blogs, stock prices, weather etc.
- Implementing user-ids and sessions (see Security lecture) can allow you to have private chats between two users.
- Socket.io allows you to group sockets into namespaces and rooms, which allows you to control who can access and post messages.

```
from flask_socketio import join_room, leave_room

@socketio.on('join')
def on_join(data):
    username = data['username']
    room = data['room']
    join_room(room)
    send(username + ' has entered the room.', room=room)

@socketio.on('leave')
def on_leave(data):
    username = data['username']
    room = data['room']
    leave_room(room)
    send(username + ' has left the room.', room=room)
```

Other

Group project

- For the group project, most of your website should be implemented via Jinja templates and server-side rendering.

- However, we will be giving marks for evidence that you do know how to make AJAX requests and transfer data to and from the server without reloading the page.

- The most natural place to include this is via implementing some `search` functionality when the user is looking for other users to share the data with, e.g.
  1. User types (partial) user-name in search box.
  2. AJAX request to server to return a list of other users that best match the text entered.
  3. User selects one from the list presented.

- Please do not make a single-page application!

- You can use sockets if you wish, but no specific marks for it.